

1. Practical5b.c

Develop a program that executes the equivalent of the shell command: `ls -l | grep ".c"` using `fork()`, `pipe()`, `dup2()`, and `exec()` system calls.

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int fd[2];

    pipe(fd);

    if(fork()==0)
    {
        dup2(fd[1], STDOUT_FILENO);
        close(fd[0]);
        close(fd[1]);
        execlp("ls", "ls", "-l", NULL);
    }

    if(fork()==0)
    {
        dup2(fd[0], STDIN_FILENO);
        close(fd[0]);
        close(fd[1]);
        execlp("grep", "grep", ".c", NULL);
    }

    close(fd[0]);
    close(fd[1]);

    wait(NULL);
    wait(NULL);

    return 0;
}
```

Output:

```
vishnu@VISHNU:~/linux-memory-monitor$ nano practical5b.c
vishnu@VISHNU:~/linux-memory-monitor$ gcc practical5b.c -o practical5b
vishnu@VISHNU:~/linux-memory-monitor$ ./practical5b
-rw-r--r-- 1 vishnu vishnu    158 Sep 26 04:58 address.c
-rw-r--r-- 1 vishnu vishnu    575 Sep 26 05:10 address_translation.c
-rwxr-xr-x 1 vishnu vishnu  16128 Sep 26 05:13 malloc_array
-rw-r--r-- 1 vishnu vishnu    382 Sep 26 05:12 malloc_array.c
-rw-r--r-- 1 vishnu vishnu   6053 Sep 22 15:09 memory.c
-rw-r--r-- 1 vishnu vishnu   5008 Sep 22 16:12 memory.c.save
-rw-r--r-- 1 vishnu vishnu  24935 Sep 19 05:57 memory_monitor.c
-rw-r--r-- 1 vishnu vishnu    463 Sep 26 05:06 memory_segements.c
-rw-r--r-- 1 vishnu vishnu   5758 Sep 23 03:33 memoryos.c
-rw-r--r-- 1 vishnu vishnu  14309 Sep 23 05:08 memoryproject.c
-rw-r--r-- 1 vishnu vishnu    424 Sep 26 05:09 page_fault.c
-rwxr-xr-x 1 vishnu vishnu  16216 Sep 26 05:25 practical5b
-rw-r--r-- 1 vishnu vishnu    507 Sep 26 05:25 practical5b.c
vishnu@VISHNU:~/linux-memory-monitor$ nano practical5b.c
vishnu@VISHNU:~/linux-memory-monitor$ |
```

2. Create a POSIX signal handling program that captures SIGINT, SIGTERM, and SIGUSR1. Demonstrate asynchronous event handling and explain the role of signal handlers.

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void handler(int sig)
{
    if(sig==SIGINT)
        printf("\nSIGINT Received\n");

    if(sig==SIGTERM)
        printf("\nSIGTERM Received\n");

    if(sig==SIGUSR1)
        printf("\nSIGUSR1 Received\n");
}

int main()
{
    signal(SIGINT,handler);
    signal(SIGTERM,handler);
    signal(SIGUSR1,handler);

    while(1)
    {
        printf("Running...\n");
        sleep(3);
    }

    return 0;
}
```

Ex: kill -SIGUSR1 PID

Ex: kill -SIGTERM PID

Output:

[illegible]